

INDUSENSE · DEEP LEARNING · B6

Détection d'Anomalies Visuelles sans images défectueuses annotées

Auto-encodeur CNN sur MVTec AD — préparation des données et stratégie d'augmentation

Partie 1 — Données

CONTEXTE

Pourquoi ne pas utiliser YOLO ?

PO

PRODUCT OWNER

On voudrait repérer automatiquement les défauts visuels sur les pièces produites. Le problème : on n'a **aucune image de pièce défectueuse annotée**. Tu as une idée ?

DEVELOPER

DEV

Oui — il faut distinguer deux familles d'approches. La première (YOLO / classification supervisée) est très performante mais nécessite **des milliers d'exemples annotés par type de défaut** — irréaliste ici. La seconde (auto-encodeur) n'a besoin **que de pièces saines**.

X NON RETENU

Classification supervisée (YOLO)

- Localisation précise des défauts
- Milliers d'exemples annotés requis
- Annotation manuelle par type de défaut
- Irréaliste sans données initiales

✓ RETENU

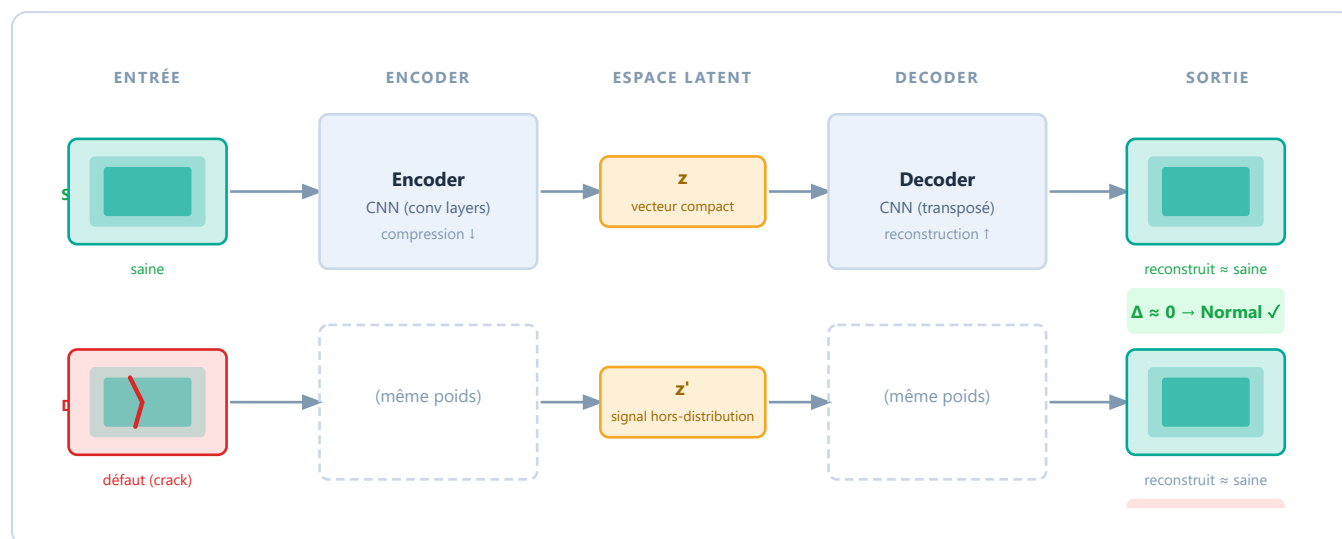
Reconstruction non supervisée (auto-encodeur)

- Aucun défaut annoté requis
- Entraînement sur pièces saines uniquement
- Adapté au démarrage (peu de données)
- Performance dépend du seuil de détection

02 — PRINCIPE

Comment fonctionne l'auto-encodeur ?

On entraîne le modèle à **compresser puis reconstruire uniquement des pièces saines**. Une fois entraîné, il ne sait reconstruire que ce qu'il a vu — c'est-à-dire le normal. Quand on lui présente un défaut, la reconstruction est mauvaise : **l'erreur de reconstruction devient le score d'anomalie**.



INSIGHT CLÉ

L'auto-encodeur ne peut reconstruire que ce qu'il a appris à reconstruire : des pièces **saines**. Présenté à un défaut, il le reconstruit "corrigé" → l'erreur pixel révèle la zone anormale.

03 — DATASET

MVTec AD — catégorie **bottle**

209 TRAIN/GOOD	83 IMAGES TEST	63 MASQUES GT	3 TYPES DE DÉFAUT
--------------------------	--------------------------	-------------------------	-----------------------------

```

bottle/ |— train/ |— good/ # 209 images saines → entraînement auto-encodeur |—
test/ |— good/ # 20 images saines → évaluation |— broken_large/ # 20 défauts →
grands éclats |— broken_small/ # 22 défauts → petits éclats |— contamination/ #
21 défauts → souillures |— ground_truth/ |— broken_large/ # masques binaires pixel
|— broken_small/ |— contamination/

```

04 — ÉTAPE 1

Chargement, normalisation et split

Pourquoi normaliser dans [0, 1] ?

Les pixels bruts sont en [0, 255] (uint8). Diviser par 255 les place dans [0, 1] en float32 — ce que les

couches de convolution et la fonction d'activation sigmoïde du décodeur attendent.

Split train / val

On réserve **20 % des images saines** pour la validation, avec un tirage aléatoire fixe (seed=42).

Résultat : **167 images train, 42 images val**. La validation servira à *calibrer le seuil* en partie 2 — elle ne doit pas voir d'images défectueuses.

```
from indusense.vision.dataset import load_train_val, load_test

# Split reproductible : 80 % train / 20 % val
X_train, X_val = load_train_val(BOTTLE_ROOT, val_ratio=0.2, seed=42)
X_test, y_test, classes = load_test(BOTTLE_ROOT)

# X_train : (167, 128, 128, 3) float32 - valeurs dans [0, 1]
# X_val    : (42, 128, 128, 3) float32
# X_test   : (83, 128, 128, 3) y_test : 0=saine / 1=défaut
```

RÈGLE IMPORTANTE

Les images de **validation et de test ne sont jamais augmentées**. L'augmentation s'applique uniquement à X_train pendant l'entraînement.

05 — ÉTAPE 2

Pipeline d'augmentation (Albumentations)

L'augmentation artificialise la variabilité des données saines pour empêcher l'auto-encodeur de mémoriser exactement les 167 images d'entraînement. Elle doit **élargir la notion de "normal" sans la dénaturer**.

✓ RETENUES POUR BOTTLE

✓ RETENU · P=0.50

HorizontalFlip

La bouteille est symétrique gauche/droite — une bouteille vue en miroir reste une bouteille saine.

✓ RETENU · P=0.70

Rotate(limit=15°)

Légère inclinaison de pose caméra ou de placement sur le convoyeur. Au-delà de 30° la bouteille sort du cadre.

✓ RETENU · P=0.70

RandomBrightnessContrast

Variation d'éclairage industriel entre sessions (±15 %) — très fréquente en production réelle.

✓ RETENU · P=0.50

HueSaturationValue

Léger drift de teinte entre lots de production (verre légèrement teinté). Amplitudes conservatrices.

✓ **RETENU** · **P=0.30**

GaussNoise

Simule le bruit capteur de la caméra industrielle.
Probabilité basse pour ne pas bruite le signal utile.

✗ ÉCARTÉES

✗ **ÉCARTÉ**

VerticalFlip

Une bouteille retournée tête en bas n'est pas "normale" — le modèle apprendrait à tolérer ce cas.

✗ **ÉCARTÉ**

ElasticTransform

Déformation irréaliste sur un objet rigide en verre.
Risque de rendre le modèle tolérant à des défauts de forme.

✗ **ÉCARTÉ**

RandomCrop / Zoom fort

Couper la bouteille produit des images hors-distribution — le modèle apprendrait à reconstruire des demi-bouteilles.

06 — POINT DE RÉFLEXION

Objet centré vs Texture — une stratégie différente

Le bon choix d'augmentation dépend **de la nature de l'objet**, pas du dataset en général. La même transform peut être excellente pour une texture et catastrophique pour un objet.



bottle — Objet centré

La bouteille est **orientée, centrée, alignée** dans l'image. Chaque transformation géométrique forte élargit la notion de "normal" vers des configurations irréelles.

HorizontalFlip ✓ **Rotate ±15°** ✓

VerticalFlip ✗ **Rotation 90°** ✗ **Elastic** ✗

Stratégie : conservatrice — favorise la sensibilité



tile — Texture isotrope

Le carrelage est **uniforme dans toutes les directions**. N'importe quelle rotation ou flip produit une image réaliste — la texture n'a pas de "haut" ou de "bas".

HorizontalFlip ✓ **VerticalFlip** ✓

Rotation 90° ✓ **RandomCrop** ✓ **Elastic** ✓

Stratégie : agressive — la texture supporte toutes